EDUARDO KALUF

K-ORCH: FERRAMENTA PARA CONTROLE E MANEJO DE SISTEMAS MULTIKERNEL ORQUESTRAÇÃO E OBSERVABILIDADE

(*versão pré-defesa, compilada em 11 de outubro de 2026*)

Tese apresentada como requisito parcial à obtenção do grau de Doutor em Ciência da Computação no Programa de Pós-Graduação em Informática, Setor de Ciências Exatas, da Universidade Federal do Paraná.

Área de concentração: *Computação*.

Orientador: Carlos Alberto Maziero.

CURITIBA

2026

SUMÁRIO

1 TRABALHOS RELACIONADOS 3
1.1 ARQUITETURAS COM MÚLTIPLOS KERNELS 3
1.2 ORQUESTRAÇÃO E BALANCEAMENTO DE CARGA 5
1.3 OBSERVABILIDADE E MONITORAMENTO 6
1.4 DISPONIBILIDADE E ISOLAMENTO 7
1.5 SUMÁRIO E TRABALHOS REVISADOS 8
1.6 CONSIDERAÇÕES FINAIS 11
REFERÊNCIAS 13

1 TRABALHOS RELACIONADOS

O aumento incessante do número de núcleos por processador em conjunto com a crescente heterogeneidade dos hardwares utilizados acabam por expor limites e dores das arquiteturas dos sistemas operacionais monolíticos, onde um único kernel é responsável por fazer o manejo de todas as estruturas de dados e controle da máquina. Desde a década de 1990, uma linha de pesquisa propõe a alternativa de executar múltiplas instâncias de um mesmo kernel, ou subconjunto de kernels, com diferentes configurações em uma mesma máquina física, onde todos funcionam de forma parecida com um sistema distribuído.

Essa maneira de tratar o SO hoje é conhecida como multikernel. Este capítulo apresenta os trabalhos correlatos estudados, mostrando a evolução da arquitetura multikernel e conectando outros trabalhos ao foco principal da pesquisa: desenvolver uma forma de orquestrar, monitorar e balancear cargas de trabalho entre diferentes nós que utilizam o sistema multikernel de maneira escalável e previsível.

Este capítulo organiza os trabalhos correlatos em quatro seções. A Seção 1.1 apresenta a evolução das arquiteturas com múltiplos kernels, desde o início com os sistemas para multiprocessadores de memória compartilhada até multikernels reduzidos utilizados em ambientes HPC. A Seção 1.2 discute como esses sistemas e orquestradores decidem a alocação de recursos e o balanceamento de carga dentro da própria arquitetura e em ambientes distribuídos. A Seção 1.3 aborda monitoramento, o qual é o critério para a tomada das decisões de orquestração. Já a Seção 1.4 trata sobre isolamento e disponibilidade dessas instâncias e quais seus objetivos em geral. Por fim, a Seção 1.5 sintetiza rapidamente tudo o que foi discutido.

1.1 ARQUITETURAS COM MÚLTIPLOS KERNELS

A ideia de estruturar o SO como um conjunto de kernels independentes surgiu há algumas décadas. O Hive (Chapin et al., 1995), organiza o SO como um sistema distribuído interno de kernels, os quais são denominados células, cada uma delas é responsável por um conjunto fixo de nós e opera como um SO independente, enquanto cooperam para oferecer às aplicações uma imagem única e transparente de sistema. A comunicação entre células ocorre por meio de *remote procedure calls* (RPC) e por leitura direta da memória de outras células.

Em vez de reescrever o SO por completo, Disco (Bugnion et al., 1997) ataca o problema inserindo um pequeno monitor de máquinas virtuais que executa várias cópias de um SO na mesma máquina, comunicando-se por protocolos distribuídos comuns como NFS e TCP/IP. A abordagem permite executar sistemas operacionais especializados lado a lado com sistemas comerciais.

O Cellular Disco (Govil et al., 2000) estende a proposta dividindo o próprio monitor em partes menores, a fim de conter falhas de hardware, transformando o multiprocessador no que seria um cluster virtual.

Já o sistema Corey (Boyd-Wickizer et al., 2008) parte do princípio de que as aplicações devem ser responsáveis por controlar o compartilhamento. Por padrão, cada estrutura de dados do kernel é usada por um único núcleo, e abstrações como *address ranges*, *kernel cores* e *shares* permitem à aplicação indicar explicitamente o que deve ser compartilhado. Por mais que execute um único kernel, o sistema Corey foi uma base importante para trabalhos futuros como Barrelfish.

Enquanto isso, o FOS (Wentzlaff e Agarwal, 2009) propõe fatorar os serviços do SO em frotas de servidores distribuídos espacialmente pelos núcleos, permitindo a comunicação exclusivamente por mensagens, isso foi motivado pela constatação de que o alocador de páginas físicas do Linux não escalava para além de oito núcleos. Além disso, essa ideia da comunicação forte por mensagens obteve grande aceitação por futuras propostas de multikernels como a implementação em (Wang, 2025b) feita a partir do kernel do linux.

A formulação moderna do conceito é apresentada por Baumann et al. (2009), que propõem tratar a máquina como uma rede de núcleos independentes. Responsáveis por desenvolver o sistema Barrelfish, onde é executado um kernel por núcleo e três princípios básicos são seguidos: comunicação entre núcleos sempre é explícita, sua estrutura é neutra em relação ao hardware e o estado é replicado em vez de ser compartilhado, sendo mantido consistente por processos monitores. Em um trabalho anterior do mesmo grupo, Schüpbach et al. (2008) é argumentado que a diversidade de hardware, nas formas de não uniformidade de memória, heterogeneidade de núcleos e diversidade entre sistemas, será um desafio tão grande quanto a escalabilidade no futuro.

No ambiente HPC, a motivação principal é combinar o baixo overhead de um kernel reduzido, comumente denotado como kernel leve, com a compatibilidade do Linux. FusedOS (Park et al., 2012) particiona os núcleos entre o Linux e uma biblioteca derivada de kernel leve CNK, executada como um processo de usuário, responsável por gerenciar núcleos dedicados às aplicações.

O trabalho Interface for Heterogeneous Kernels (IHK) (Shimosawa et al., 2014) generaliza essa estrutura em um modelo mestre-escravo, no qual o Linux, como kernel mestre, reserva núcleos e memória, inicializa, consulta o estado, desliga e destrói instâncias de kernels leves, que se comunicam por uma camada própria.

Ao comparar seis desses projetos, Gerofi et al. (2016) optam por distinguir esses sistemas com particionamento estático de recursos, de sistemas que permitem reparticionamento dinâmico, dando grande importância a este ponto.

Além disso, duas propostas exploram áreas complementares que valem a pena serem citadas. PIKA (Beckmann et al., 2014) projeta uma pilha de rede específica para multikernels, dividindo em servidores que se comunicam por mensagens. Enquanto MultiK (Kuo et al., 2019), que apesar do nome não executa múltiplos kernels independentes, mas sim oferece a cada

aplicação uma cópia do código do kernel que contém apenas o necessário para ela executar. Isso ocorre automaticamente durante a troca de contexto dos processos, tendo o objetivo de reduzir a superfície de ataque de aplicações que não precisam de todos os elementos do sistema e dando ênfase na ideia de especializar o kernel por aplicação.

1.2 ORQUESTRAÇÃO E BALANCEAMENTO DE CARGA

Decidir onde executar cada carga e como redistribuí-la conforme a demanda é um problema estudado há muito tempo em sistemas distribuídos. Casavant e Kuhl (1988) propõem uma taxonomia hierárquica de escalonamento em sistemas distribuídos que distingue abordagens estáticas de dinâmicas, Além disso, as abordagens dinâmicas são classificadas quanto à distribuição da responsabilidade pelas decisões e à cooperação entre os nós.

Shivaratri et al. (1992) decompõem os algoritmos de distribuição de carga em quatro principais políticas, de transferência, decidir quando mover uma tarefa, de seleção, qual tarefa mover, localização, para onde mover e de informação, quando coletar o estado dos nós. Os algoritmos ainda são classificados naqueles que são iniciados pelo emissor, pelo receptor ou simétricos.

Dois resultados fundamentam políticas simples, mas eficientes e escaláveis de balanceamento. Primeiro, Mitzenmacher (2001) mostra que escolhendo a opção menos carregada entre apenas dois destinos aleatórios reduz exponencialmente a carga máxima esperada em relação a simplesmente escolher aleatoriamente. Em outro estudo, Blumofe e Leiserson (1999) provam que o roubo de tarefas de maneira aleatorizada de um processador ativo para outro ocioso, executa computações com tempo próximo do ótimo.

Ao revisar orquestradores como Kubernetes, Docker Swarm e Apache Mesos, Swetha et al. (2025) identificam a escassez de propostas de orquestração orientada pela utilização de recursos e comparam a adequação de cada ferramenta. Na mesma linha, Lu et al. (2025) propõem uma implantação de contêineres em dois estágios: um algoritmo guloso distribui contêineres entre máquinas virtuais com granularidade grossa enquanto outro algoritmo genético reparte os recursos dentro de cada uma das máquinas virtuais.

Agora em sistemas com múltiplos kernels, desde o início, era necessário saber como distribuir os recursos da máquina entre as instâncias. No Hive, os mecanismos de compartilhamento de recursos são implementados pela cooperação entre as células, mas a política é atribuída a um processo de usuário, o único com visão global do sistema, responsável por decidir de quais células alocar memória e como escalonar processos (Chapin et al., 1995).

No Disco, o monitor aplica políticas globais sobre as máquinas virtuais, movendo memória entre elas para evitar paginação desnecessária e escalonando dinamicamente as vCPUs para equilibrar a carga (Bugnion et al., 1997). O Cellular Disco (Govil et al., 2000) apresenta uma solução mais elaborada para sistemas multiprocessadores, nele, cada processador mantém sua própria fila de processadores virtuais, combinando um balanceador ocioso, que usa apenas

informação local, com um balanceador periódico, que consulta uma árvore de carga global atualizada periodicamente.

No Barrelfish, a base de conhecimento do sistema reúne informação pelo hardware e medições em tempo de execução, respondendo a consultas de otimização, como a alocação de drivers a núcleos e a construção de árvores de mensagens adaptadas à máquina (Baumann et al., 2009; Schüpbach et al., 2008).

Nos multikernels com foco em HPC, a gerência das instâncias tende a ser exposta como uma interface. A IHK oferece operações para reservar núcleos e memória, inicializar, consultar, desligar e destruir instâncias de kernels leves, porém, não define políticas sobre quando as utilizar (Shimosawa et al., 2014).

No projeto Hobbes, Kocoloski et al. (2015) é proposto a execução em enclaves distintos de um mesmo nó, com um gerenciador de nó responsável pelo ciclo de vida dos enclaves e mecanismos de memória compartilhada entre os mesmos.

Mais recentemente, a proposta de suporte a múltiplas instâncias do Linux em uma mesma máquina (Wang, 2025b) utiliza a ideia de carregar as instâncias via *kexec*, recebendo núcleos e memória que podem ser realocados em tempo de execução por meio de uma *device tree* e do mecanismo de *hotplug* (Wang, 2025a). A ferramenta kerf (Multikernel Technologies, 2025) ficou responsável por gerenciar a criação, configuração, monitoramento e a destruição dessas instâncias.

1.3 OBSERVABILIDADE E MONITORAMENTO

Esta seção apresenta fundamentos de como observar, coletar dados e analisá-los em sistemas distribuídos, em seguida como trabalhos aplicam técnicas parecidas em ambientes com múltiplos kernels.

Mansouri-Samani e Sloman (1993) definem um modelo funcional de monitoramento de sistemas distribuídos dividido em quatro atividades (geração, processamento, disseminação e apresentação de informação), tratadas como blocos que se combinam em diferentes proporções e momentos da análise, sendo uma base essencial para os trabalhos que seguem. Além disso, apontam a desatualização da informação como o principal problema existente dentro do monitoramento.

Em uma revisão recente voltada a arquitetura de microsserviços, Faseeha et al. (2025) analisam ferramentas de observabilidade em diferentes níveis, como de sistema, de serviço e de rede, organizadas em torno de três fontes que hoje já são comumente disseminadas: *metrics*, *logs* e *traces*.

Para métricas em clusters, Ganglia (Massie et al., 2004) adotam uma organização hierárquica: dentro de cada cluster, os nós comunicam e trocam seus estados via *multicast*. Clusters distintos são federados por uma árvore de conexões *peer-to-peer*, com armazenamento

em bases de séries temporais. O projeto privilegia baixa sobrecarga por nó e escalabilidade, ao custo de métricas agregadas e de uma menor granularidade temporal.

Agora, em um ambiente multikernel, a IHK concentra a gerência no kernel *master* (Shimosawa et al., 2014), responsável por tomar o controle do mapeamento da memória física e orquestrar a alocação e reserva de recursos para os kernels *slave* do ambiente.

O LDMS (Agelastos et al., 2014) mostram que é possível monitorar continuamente um sistema inteiro, com a frequência adequada, mesmo em ambientes HPC. LDMS utiliza processos coletores nos nós e processos agregadores, que reúnem os dados. Os autores quantificam a sobrecarga do monitoramento sobre as aplicações como baixa e negligenciável.

O Omni-Kernel (Kvalnes et al., 2015) medem o consumo de recursos por completo, através de um processo, uma máquina virtual ou uma transação. Nele, o kernel é fatorado em componentes que se comunicam por mensagens, e escalonadores existem no meio do caminho, responsáveis por controlar a ordem de processamento e contabilizar o consumo.

Em uma proposta de hardware, West et al. (2009) dedicam um núcleo específico ao monitoramento, alimentado por detectores de eventos distribuídos pelo processador, mostrando como uma instância de kernel poderia contabilizar e exportar seu próprio consumo.

Os próprios sistemas com múltiplos kernels também possuem sistemas de monitoramento e balanceamento por si só. A base de conhecimento do Barrelfish incorpora medições em tempo de execução, como a latência de comunicação entre cada par de núcleos (Baumann et al., 2009), o balanceador periódico do Cellular Disco depende de uma árvore de carga atualizada constantemente (Govil et al., 2000) e o PIKA mede o tempo de serviço de cada aplicação para orientar as escolhas realizadas (Beckmann et al., 2014). Porém, a informação é coletada e consumida numa única máquina, não sendo explorado a ideia do compartilhamento entre nós em um cluster.

1.4 DISPONIBILIDADE E ISOLAMENTO

Uma grande motivação para dividir o sistema em múltiplos kernels é a disponibilidade das aplicações. No Hive (Chapin et al., 1995), uma falha de hardware ou de software afeta apenas os processos que usam os recursos da célula prejudicada. O sistema emprega uma proteção em cada página contra escritas de outras células, descarta de maneira preventiva as páginas que uma célula que falhou poderia ter corrompido e possui detecção de falhas por heurísticas, cuja confirmação exige consenso das células sobreviventes.

O Disco trata cada máquina virtual como unidade de contenção de falhas de software (Bugnion et al., 1997), e o Cellular Disco estende essa contenção a falhas de hardware. Em centenas de experimentos com injeção de falhas, a recuperação foi bem-sucedida em todos os casos, levando menos de meio segundo (Govil et al., 2000).

Além de conter falhas, a divisão em múltiplos kernels também é usada para isolar o desempenho das aplicações. O Pisces (Ouyang et al., 2015) permite a coexistência, em um

mesmo nó, de múltiplos ambientes de SO independentes e isolados, denominados enclaves, cada um executando um co-kernel leve, capaz de gerenciar uma partição de recursos de hardware, tratando o isolamento de desempenho como requisito principal e tendo seu foco em HPC

Com ênfase na parte de segurança, pesquisas como BlackBox (Van't Hof e Nieh, 2022) buscam proteger a confidencialidade e a integridade de contêineres sem confiar no SO, por meio de um monitor que cria espaços de endereçamento físico protegidos para cada contêiner. Enquanto o MultiK (Kuo et al., 2019) garante maior segurança reduzindo a superfície de ataque ao remover mais de 90% do código do kernel visível a cada aplicação que está sendo executada. Esses trabalhos, por mais que não executados sob a arquitetura multikernel, ressoam fortemente com sua filosofia.

1.5 SUMÁRIO E TRABALHOS REVISADOS

A Tabela 1.1 apresenta os trabalhos selecionados e descritos, além de apresentar um resumo sobre as principais contribuições e foco de cada um deles.

Tabela 1.1 – Resumo dos Trabalhos Relacionados.

Referência	**Categoria**	**Contribuição principal**	**Relação com este trabalho**
Chapin et al. (1995)	Multikernel	Hive: células com kernels independentes, contenção de falhas e política global no Wax	Origem da separação entre mecanismo e política
Bugnion et al. (1997)	Multikernel	Disco: múltiplas cópias de SO sobre monitor de máquinas virtuais enxuto	Políticas globais sobre instâncias independentes
Govil et al. (2000)	Multikernel	Cellular Disco: monitor dividido em células, cluster virtual com contenção de falhas	Balanceamento hierárquico entre células
Boyd-Wickizer et al. (2008)	Multikernel	Corey: aplicação controla o compartilhamento de estruturas do kernel	Mecanismos de alocação sem política automática
Wentzlaff e Agarwal (2009)	Multikernel	fos: serviços do SO em frotas de servidores distribuídos por mensagens	Posicionamento espacial de serviços

Continua na próxima página

Resumo dos Trabalhos Relacionados. (Continuação)

Referência	**Categoria**	**Contribuição principal**	**Relação com este trabalho**
Baumann et al. (2009)	Multikernel	Barrelfish: um kernel por núcleo, mensagens explícitas e estado replicado	Define a arquitetura multikernel
Schüpbach et al. (2008)	Multikernel	Diversidade de hardware e base de conhecimento do sistema	Base de informação para decisões de alocação
Park et al. (2012)	Multi-kernel HPC	FusedOS: kernel leve em nível de usuário ao lado do Linux	Partição estática entre kernels
Shimosawa et al. (2014)	Multi-kernel HPC	IHK: interface de ciclo de vida e comunicação entre kernels	Plano de controle local a um nó
Gerofi et al. (2016)	Multi-kernel HPC	Levantamento de seis multi-kernels e critérios de comparação	Particionamento estático versus dinâmico
Beckmann et al. (2014)	Multikernel	PIKA: pilha de rede dividida em servidores com balanceamento distribuído	Balanceamento sem memória compartilhada
Kuo et al. (2019)	Especialização de kernel	MultiK: visão do código do kernel especializada por aplicação	Especialização por carga, não é multikernel
Casavant e Kuhl (1988)	Balanceamento de carga	Taxonomia de escalonamento estático e dinâmico em sistemas distribuídos	Base de classificação das políticas
Shivaratri et al. (1992)	Balanceamento de carga	Políticas de transferência, seleção, localização e informação	Estrutura de projeto do balanceador
Mitzenmacher (2001)	Balanceamento de carga	Poder de duas escolhas no balanceamento aleatorizado	Decisões descentralizadas com pouca informação
Blumofe e Leiserson (1999)	Balanceamento de carga	Roubo de tarefas aleatorizado com garantias de desempenho	Balanceamento iniciado pelo receptor

Continua na próxima página

Resumo dos Trabalhos Relacionados. (Continuação)

Referência	**Categoria**	**Contribuição principal**	**Relação com este trabalho**
Swetha et al. (2025)	Orquestração	Revisão de orquestradores de contêineres e lacuna da orquestração por utilização	Lacuna análoga no nível de contêineres
Lu et al. (2025)	Balanceamento de carga	Implantação em dois estágios: guloso entre VMs e genético dentro da VM	Estrutura em dois níveis, avaliada em simulação
Kocoloski et al. (2015)	Orquestração multikernel	Hobbes: composição de aplicações entre enclaves de um nó	Gerência de enclaves restrita a um nó
Wang (2025b)	Orquestração multikernel	Proposta de múltiplas instâncias do Linux via `kexec`	Plataforma-alvo provável do orquestrador
Wang (2025a)	Orquestração multikernel	Realocação dinâmica de recursos entre instâncias via *device tree*	Mecanismo de redimensionamento de instâncias
Multikernel Technologies (2025)	Orquestração multikernel	kerf: ciclo de vida e recursos de instâncias de kernel	Ferramenta mais próxima, limitada a um hospedeiro
Mansouri-Samani e Sloman (1993)	Observabilidade	Modelo funcional de monitoramento de sistemas distribuídos	Base para a coleta de informação
Faseeha et al. (2025)	Observabilidade	Revisão de observabilidade em microsserviços com métricas, registros e rastros	Fundamentos e terminologia da área
Massie et al. (2004)	Observabilidade	Ganglia: monitoramento hierárquico e federado de clusters	Agregação de métricas por nó e entre clusters
Agelastos et al. (2014)	Observabilidade multikernel	LDMS: monitoramento contínuo e leve de sistemas de grande escala	Modelo de coleta por nó e agregação em HPC

Continua na próxima página

Resumo dos Trabalhos Relacionados. (Continuação)

Referência	**Categoria**	**Contribuição principal**	**Relação com este trabalho**
Kvalnes et al. (2015)	Observabilidade multikernel	Omni-Kernel: medição e atribuição de todo consumo de recursos	Contabilização por atividade
West et al. (2009)	Observabilidade multikernel	Núcleo de hardware dedicado ao monitoramento de desempenho	Realimentação do monitoramento ao escalonador
Erlingsson et al. (2011)	Observabilidade multikernel	Fay: rastreamento dinâmico do kernel ao cluster	Une kernel e cluster, mas com um kernel por máquina
Ferreira et al. (2008)	Observabilidade multikernel	Sensibilidade de aplicações ao ruído do SO por injeção no kernel	Coleta não pode interferir nos núcleos da aplicação
Ouyang et al. (2015)	Isolamento multikernel	Pisces: enclaves com cokernels leves e isolamento de desempenho	Isolamento de desempenho entre instâncias
Van't Hof e Nieh (2022)	Isolamento	BlackBox: monitor que isola contêineres de um SO não confiável	Imposição de fronteiras sem hipervisor completo

1.6 CONSIDERAÇÕES FINAIS

A análise de todos os trabalhos revela uma trajetória de algumas décadas com uma característica em comum, a gerência da arquitetura multikernel é sempre feita no escopo de uma única máquina.

É possível evidenciar, principalmente três limitações, que abrem diversas portas para avanço e pesquisa. Em primeiro lugar, como dito anteriormente, o escopo de gerência é sempre um único nó, no projeto Hive e no Disco é mencionado clusterização como extensão futura, mas nenhum dos trabalhos analisados de fato as realiza. Em segundo lugar, os mecanismos frequentemente existem apenas na parte teórica, como o Wax que nunca foi de fato implementado (Chapin et al., 1995), ou deixam as decisões diretamente à aplicação, ou usuário, como é feito no Corey e no IHK (Boyd-Wickizer et al., 2008; Shimosawa et al., 2014). Em terceiro lugar, as ferramentas de observabilidade tomam como unidade a máquina, o contêiner ou a requisição (Massie et al., 2004; Agelastos et al., 2014; Erlingsson et al., 2011), e não a instância de kernel.

Entre os trabalhos analisados, nenhum trata a instância de kernel como unidade escalonável, elastica, observável e recuperável num conjunto amplo de nós. Sendo assim, existe espaço para a construção de uma camada de orquestração, tomando ideias de orquestradores de contêineres comerciais da

atualidade e estendendo-as para um cluster de nós em ambientes com a arquitetura multikernel. Realizando o monitoramento das instâncias, coletando informações dos núcleos e as agregando entre nós, para ser possível a tomada de decisão da criação de novos jobs, redimensionamento e redistribuição de cargas pelo cluster.

REFERÊNCIAS

Agelastos, A., Allan, B., Brandt, J., Cassella, P., Enos, J., Fullop, J., Gentile, A., Monk, S., Naksinehaboon, N., Ogden, J., Rajan, M., Showerman, M., Stevenson, J., Taerat, N. e Tucker, T. (2014). The lightweight distributed metric service: A scalable infrastructure for continuous monitoring of large scale computing systems and applications. Em *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, páginas 154–165, New Orleans - LA, USA.

Baumann, A., Barham, P., Dagand, P.-E., Harris, T., Isaacs, R., Peter, S., Roscoe, T., Schüpbach, A. e Singhania, A. (2009). The multikernel: A new OS architecture for scalable multicore systems. Em *22nd ACM Symposium on Operating Systems Principles (SOSP)*, páginas 29–44, Big Sky - MT, USA.

Beckmann, N. Z., Gruenwald III, C., Johnson, C. R., Kasture, H., Sironi, F., Agarwal, A., Kaashoek, M. F. e Zeldovich, N. (2014). PIKA: A network service for multikernel operating systems. Relatório Técnico MIT-CSAIL-TR-2014-002, MIT Computer Science and Artificial Intelligence Laboratory, Cambridge - MA, USA.

Blumofe, R. D. e Leiserson, C. E. (1999). Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748.

Boyd-Wickizer, S., Chen, H., Chen, R., Mao, Y., Kaashoek, F., Morris, R., Pesterev, A., Stein, L., Wu, M., Dai, Y., Zhang, Y. e Zhang, Z. (2008). Corey: An operating system for many cores. Em *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, páginas 43–57, San Diego - CA, USA.

Bugnion, E., Devine, S., Govil, K. e Rosenblum, M. (1997). Disco: Running commodity operating systems on scalable multiprocessors. *ACM Transactions on Computer Systems*, 15(4):412–447.

Casavant, T. L. e Kuhl, J. G. (1988). A taxonomy of scheduling in general-purpose distributed computing systems. *IEEE Transactions on Software Engineering*, 14(2):141–154.

Chapin, J., Rosenblum, M., Devine, S., Lahiri, T., Teodosiu, D. e Gupta, A. (1995). Hive: Fault containment for shared-memory multiprocessors. Em *15th ACM Symposium on Operating Systems Principles (SOSP)*, páginas 12–25, Copper Mountain - CO, USA.

Erlingsson, Ú., Peinado, M., Peter, S. e Budiu, M. (2011). Fay: Extensible distributed tracing from kernels to clusters. Em *23rd ACM Symposium on Operating Systems Principles (SOSP)*, páginas 311–326, Cascais - Portugal.

Faseeha, U., Syed, H. J., Samad, F., Zehra, S. e Ahmed, H. (2025). Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. *IEEE Access*, 13:72011–72039.

Ferreira, K. B., Bridges, P. e Brightwell, R. (2008). Characterizing application sensitivity to OS interference using kernel-level noise injection. Em *ACM/IEEE Conference on Supercomputing (SC)*, Austin - TX, USA.

Gerofi, B., Ishikawa, Y., Riesen, R., Wisniewski, R. W., Park, Y. e Rosenburg, B. (2016). A multi-kernel survey for high-performance computing. Em 6th *International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, Kyoto - Japan.

Govil, K., Teodosiu, D., Huang, Y. e Rosenblum, M. (2000). Cellular Disco: Resource management using virtual clusters on shared-memory multiprocessors. *ACM Transactions on Computer Systems*, 18(3):229–262.

Kocoloski, B., Lange, J., Abbasi, H., Bernholdt, D. E., Jones, T. R., Dayal, J., Evans, N., Lang, M., Lofstead, J., Pedretti, K. e Bridges, P. G. (2015). System-level support for composition of applications. Em 5th *International Workshop on Runtime and Operating Systems for Supercomputers (ROSS)*, Portland - OR, USA.

Kuo, H.-C., Gunasekaran, A., Jang, Y., Mohan, S., Bobba, R. B., Lie, D. e Walker, J. (2019). MultiK: A framework for orchestrating multiple specialized kernels. https://arxiv.org/abs/1903.06889. arXiv:1903.06889. Acessado em 10/10/2026.

Kvalnes, Å., Johansen, D., van Renesse, R., Schneider, F. B. e Valvåg, S. V. (2015). Omni-Kernel: An operating system architecture for pervasive monitoring and scheduling. *IEEE Transactions on Parallel and Distributed Systems*, 26(10):2849–2862.

Lu, C., Zhou, J. e Zou, Q. (2025). An optimized approach for container deployment driven by a two-stage load balancing mechanism. *PLOS ONE*, 20(1). Artigo e0317039.

Mansouri-Samani, M. e Sloman, M. (1993). Monitoring distributed systems. *IEEE Network*, 7(6):20–30.

Massie, M. L., Chun, B. N. e Culler, D. E. (2004). The Ganglia distributed monitoring system: Design, implementation, and experience. *Parallel Computing*, 30(7):817–840.

Mitzenmacher, M. (2001). The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104.

Multikernel Technologies (2025). kerf: Multikernel management system. https://github.com/multikernel/kerf. Acessado em 10/10/2026.

Ouyang, J., Kocoloski, B., Lange, J. R. e Pedretti, K. (2015). Achieving performance isolation with lightweight co-kernels. Em *24th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, páginas 149–160, Portland - OR, USA.

Park, Y., Van Hensbergen, E., Hillenbrand, M., Inglett, T., Rosenburg, B., Ryu, K. D. e Wisniewski, R. W. (2012). FusedOS: Fusing LWK performance with FWK functionality in a heterogeneous environment. Em *24th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, páginas 211–218, New York - NY, USA.

Schüpbach, A., Peter, S., Baumann, A., Roscoe, T., Barham, P., Harris, T. e Isaacs, R. (2008). Embracing diversity in the Barrelfish manycore operating system. Em *Workshop on Managed Many-Core Systems (MMCS)*, Boston - MA, USA.

Shimosawa, T., Gerofi, B., Takagi, M., Nakamura, G., Shirasawa, T., Saeki, Y., Shimizu, M., Hori, A. e Ishikawa, Y. (2014). Interface for heterogeneous kernels: A framework to enable hybrid OS designs targeting high performance computing on manycore architectures. Em *21st International Conference on High Performance Computing (HiPC)*, páginas 1–10, Goa - India.

Shivaratri, N. G., Krueger, P. e Singhal, M. (1992). Load distributing for locally distributed systems. *Computer*, 25(12):33–44.

Swetha, R., Thriveni, J. e Venugopal, K. R. (2025). Resource utilization-based container orchestration: Closing the gap for enhanced cloud application performance. *SN Computer Science*, 6. Artigo 191.

Van't Hof, A. e Nieh, J. (2022). BlackBox: A container security monitor for protecting containers on untrusted operating systems. Em *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, páginas 683–700, Carlsbad - CA, USA.

Wang, C. (2025a). Multikernel: Kernel-to-kernel isolation with elastic resource management. https://lpc.events/event/19/contributions/2074/. Linux Plumbers Conference. Acessado em 10/10/2026.

Wang, C. (2025b). [RFC patch 0/7] kernel: Introduce multikernel architecture support. https://lore.kernel.org/lkml/20250918222607.186488-1-xiyou.wangcong@gmail.com/. Linux Kernel Mailing List. Acessado em 10/10/2026.

Wentzlaff, D. e Agarwal, A. (2009). Factored operating systems (fos): The case for a scalable operating system for multicores. *ACM SIGOPS Operating Systems Review*, 43(2):76–85.

West, P. E., Peress, Y., Tyson, G. S. e McKee, S. A. (2009). Core monitors: Monitoring performance in multicore processors. Em *6th ACM Conference on Computing Frontiers (CF)*, Ischia - Italy.